

Модель даних. Базові типи. Синтаксис

Рекомендовані джерела:

1. **Python 3.x documentation** — <https://docs.python.org/3/>
 2. **PEP 8** — Style Guide for Python Code. — <https://www.python.org/dev/peps/pep-0008/>
 3. Селіверстов Р., Мельничин А., **Основи програмування мовою Python**. - ЛНУ, 2020.
 4. Lambert K. A. **Fundamentals of Python: First Programs**. — Cengage, 2019.
 5. Sweigart A. **Automate the Boring Stuff with Python: Practical Programming for Total Beginners**. — No Starch Press, 2014.
 6. Wentworth P., ... **How to Think Like a Computer Scientist: Learning with Python 3**. — Green Tea Press, 2018.
-

Де використовується:

- **Веб-розробка** (найвідоміші фреймворки - Flask, Django) — Dropbox, серверні частини Instagram, Pinterest, Reddit.
 - **Наукові дослідження, Data Science** — SciPy, NumPy, Matplotlib, Pandas, Scikit-learn, TensorFlow, Keras.
 - **Десктопні застосунки** — GIMP, Blender, Ubuntu Software Center.
 - **Ігри** — скрипти взаємодії персонажів, запуску сцен, обробки подій в Battlefield 2, World of Tanks, Civilization IV, EVE Online.
 - **Розробка вбудованих систем для пристроїв** — Raspberry Pi.
 - **Реалізація мереж peer-to-peer** — BitTorrent.
 - **За замовчуванням на всіх Linux-серверах.**
-

Ким використовувався/використовується:

- **Google** — пошуковий рушій (search engine), частина YouTube.
 - **Агентство національної безпеки США** — шифрування та аналіз розвідданих.
 - **Maya, Pixar** — створення мультиплікації/анімації.
 - **Intel, Cisco, HP, Seagate, Qualcomm, IBM** — тестування.
 - **JPMorgan Chase, UBS, Getco, Citadel** — прогнозування фінансових ринків.
 - **NASA, Los Alamos, Fermilab, JPL** — наукові обчислення.
 - **iRobot** — розробка комерційних роботизованих пристроїв.
-

Особливості мови Python

- **Інтерпретована** (не компілюється, звичайний текстовий файл є виконуваним, але в системі повинен бути встановлений інтерпретатор Python). Насправді процес компіляції виконується інтерпретатором автоматично і прихований від користувача.
 - **З відкритим кодом** (можна вільно переглядати код та використовувати його).
 - **Багатоплатформність**.
-

Переваги

- компактність коду;
 - висока читабельність коду;
 - обширна стандартна бібліотека і підтримка сторонніх бібліотек;
 - швидкість розробки.
-

Недоліки

- невисока, порівняно з компільованими мовами, продуктивність;
 - глобальне блокування інтерпретатора (GIL) — у кожен момент часу в одному процесі інтерпретатора може виконуватися лише один потік коду, незалежно від кількості доступних ядер процесора.
-

Розробка

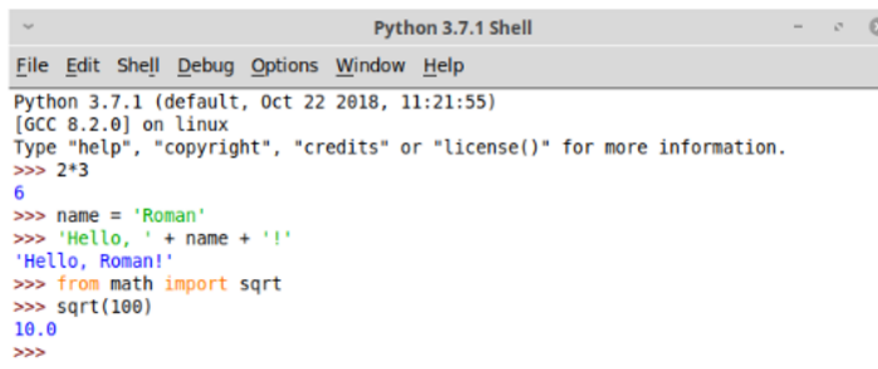
Наразі співіснують дві паралельні версії (гілки) мови:

- **Python 2** (з 2020 року не підтримується, остання версія - Python 2.7.18);
- **Python 3** (станом на 2 вересня 2023 року - Python 3.11.5).

Середовища розробки:

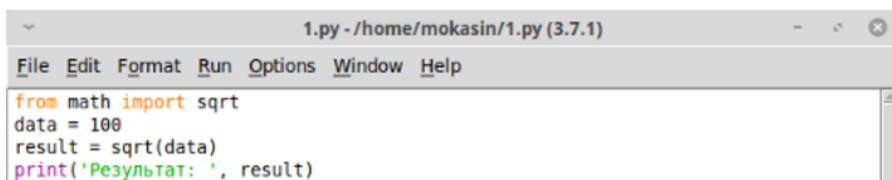
- IDLE (<https://www.python.org/>);
- Spyder IDE;
- PyCharm;
- Jupyter Notebook (дистрибутив Anaconda);
- Visual Studio Code.

Інтерактивний режим

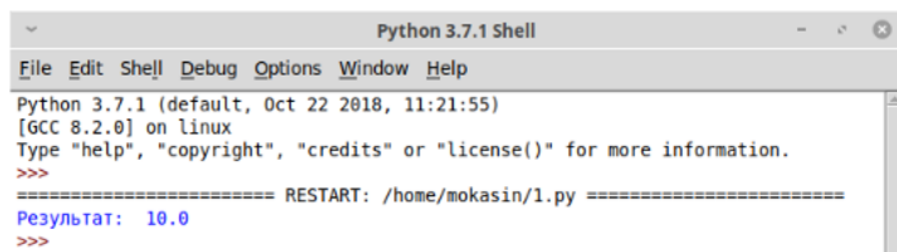


```
Python 3.7.1 Shell
File Edit Shell Debug Options Window Help
Python 3.7.1 (default, Oct 22 2018, 11:21:55)
[GCC 8.2.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>> 2*3
6
>>> name = 'Roman'
>>> 'Hello, ' + name + '!'
'Hello, Roman!'
>>> from math import sqrt
>>> sqrt(100)
10.0
>>>
```

Сценарний режим



```
1.py - /home/mokasin/1.py (3.7.1)
File Edit Format Run Options Window Help
from math import sqrt
data = 100
result = sqrt(data)
print('Результат: ', result)
```



```
Python 3.7.1 Shell
File Edit Shell Debug Options Window Help
Python 3.7.1 (default, Oct 22 2018, 11:21:55)
[GCC 8.2.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: /home/mokasin/1.py =====
Результат: 10.0
>>>
```

Модель даних

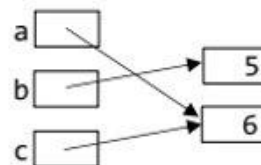
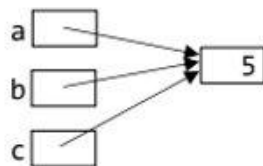
! Усе – об'єкти (використовується спільний базовий клас для всіх типів).

! Динамічна типізація: кожна змінна може набувати значення довільного типу (у змінних зберігаються посилання на дані).

! Збирач сміття (реалізовані механізми підрахунку посилань і автоматичного збирання сміття (об'єктів з нульовим лічильником посилань)).

! Використовуються посилання на ідентичні значення, щоб не створювати однакові об'єкти.

```
>>> a = 5
>>> b = a
>>> c = 5
>>> id(a)
1801612448
>>> id(b)
1801612448
>>> id(c)
1801612448
```



```
>>> c = 6
>>> id(c)
1801612480
>>> a += 1
>>> id(a)
1801612480
```

Базові (вбудовані) типи даних

	Змінювані (mutable)	Незмінювані (immutable)
Прості		int (ціле число) float (дійсне число) complex (комплексне число)
Складені	list (список) dict (словник) set (множина)	str (рядок) tuple (кортеж)

Особливості синтаксису

Коментування коду

Щоб зробити код програми зрозумілішим, рекомендовано використовувати коментарі — фрагменти коду з пояснювальними дописами, які не виконуються (пропускаються) інтерпретатором. Python дає змогу додавати однорядкові та багаторядкові коментарі.

Однорядковий коментар починається символом `#`. Усе, що слідує за цим символом до кінця рядка, вважається коментарем і не впливає на виконання програми. Багаторядковий коментар починається і закінчується трьома символами `'''` або `"""`.

```
'''Програма обчислює квадратний корінь з числа, записаного у змінну data, та
виводить отриманий результат на екран'''
from math import sqrt # імпорт функції для обчислення квадратного кореня
data = 100             # число, з якого потрібно добути корінь
result = sqrt(data)    # добування кореня
# --- Друк результату: ---
print('Результат: ', result)
```

Літерали

Літерал — постійне значення певного типу даних, записане у коді програми. Літерал можна розглядати як форму запису даних, за якою інтерпретатор однозначно визначає тип і значення цих даних.

<code>50</code>	<code># int</code>
<code>2.5</code>	<code># float</code>
<code>1-2.5j</code>	<code># complex</code>
<code>'world'</code>	<code># str</code>
<code>[1, 'two', 3.0]</code>	<code># list</code>
<code>(1, 'two', 3.0)</code>	<code># tuple</code>
<code>{'name': 'Roman', 'age': 47}</code>	<code># dict</code>
<code>{'red', 'green', 'yellow'}</code>	<code># set</code>

Оператори

Арифметичні оператори: **+**, **-**, *****, **/**, ******, **//**, **%**

Бітові оператори: **&**, **|**, **^**, **~**

Логічні оператори: **and**, **or**, **not**

Оператори порівняння: **>**, **<**, **>=**, **<=**, **==**, **!=**

Оператори перевірки на входження: **in**, **not in**

Оператор присвоєння: **=**

Один і той самий оператор може виконувати різні операції залежно від типу значень, до яких він застосовується (ця властивість називається поліморфізмом):

```
2 + 3          # 5
'2' + '3'      # '23'
'2' * 3         # '222'
'2' + 3         # TypeError: can only concatenate str (not "int") to str
2 + '3'         # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Інструкції

Інструкції — логічні блоки програми, які містять вирази і забезпечують виконання конкретних команд.

Кінцем простої інструкції слугує кінець рядка, тобто кожна проста інструкція записується в одному рядку:

```
a = 2
b = 3
print(a + b)
```

Python допускає запис кількох простих інструкцій в одному рядку, розділяючи їх крапкою з комою, але такий стиль **не притаманний** Python-програмам:

```
a = 2; b = 3; print(a + b)
```

Якщо фрагмент коду програми міститься всередині дужок (круглих, квадратних або фігурних), то він може бути записаний у кілька рядків попри те, що належить одній простій інструкції. Наприклад:

```
days = ["Понеділок", "Вівторок",
         "Середа", "Четвер", "П'ятниця",
         "Субота", "Неділя"]
```

Якщо інструкція містить вкладені інструкції, то вона називається складеною і записується у кілька рядків, використовуючи такий синтаксис: в кінці основної інструкції ставиться двокрапка, а кожна вкладена інструкція починається з обов'язкового однакового відступу, який за замовчуванням складається з чотирьох пробілів або одного символа табуляції.

Наприклад, один з варіантів реалізації умовної інструкції виглядає так:

```
if hours > 24:  
    print('Прошло більше доби')
```

Якщо вкладена інструкція у свою чергу є теж складеною інструкцією, то відступи задаються уже відносно неї:

```
if hours > 24:  
    if hours > 48:  
        print('Прошло більше двох діб')  
    else:  
        print('Прошло більше доби')  
else:  
    print('Прошло менше доби')
```

Відступи є одним з таких елементів синтаксису, який відрізняє Python від більшості інших мов програмування. Вони покращують читабельність програми і є по суті єдиним механізмом, який надає Python для групування коду. Для організації відступів, крім пробілів, можна використовувати символ табуляції. Але не допускається змішування пробілів і символів табуляції в межах одного блоку — інтерпретатор видасть синтаксичну помилку “unexpected indent”.

Змінні

1) не можна в якості імені змінної використовувати зарезервовані імена, які ще називають ключовими словами Python;

2) ім'я змінної повинно починатися з літери або символу підкреслення `_` і може складатись з довільної кількості літер, цифр і символів підкреслення (використання інших символів заборонене).

У Python немає констант! Якщо значення змінної залишається незмінним впродовж виконання усієї програми і необхідно підкреслити, що така змінна відіграє роль константи, то її іменують великими літерами, а для розділення слів використовують символ підкреслення: `SIZE`, `STANDARD_SIZE`.

Групове присвоєння

Якщо кільком змінним потрібно присвоїти однакове значення, використовують групове присвоєння:

```
x = y = z = 0
a, b, c = 1, 2, 3

y, z = a, b
a, b = b, a
```

Вбудовані функції. Методи

В Python є величезна кількість функцій. Деякі з них вбудовані, інші вимагають імпортування модулів стандартної бібліотеки або навіть інсталяції зовнішніх бібліотек. Приклад виклику функції:

```
max(-1, 0.25, 1/8)      # повертає найбільше число, тобто 0.25
```

Методи своєю поведінкою нагадують функції, але мають інший синтаксис виклику, який використовує точкову нотацію:

```
"1123101".count('1')    # повертає кількість одиниць, тобто 4
```

Щоб отримати перелік усіх доступних методів для певного типу, можна в інтерактивному режимі скористатися функцією `dir`, передавши їй назву типу або будь-який літерал цього типу:

```
>>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__',
'__eq__', '__format__', '__ge__', '__getattr__', '__getitem__',
'__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__',
'__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__',
'__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__',
'__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize',
'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find',
'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal',
'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace',
'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans',
'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex',
'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith',
'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

PEP 8

Ознайомитися самостійно з розділами PEP 8, які стосуються наступного:

- Максимальна довжина рядка - 79 символів.
 - Відокремлення оператора присвоєння (звернути увагу на передавання значень іменованих аргументів).
 - Використання пробілів у виразах.
 - Порожні рядки після імпорту, між функціями та класами.
 - Правила іменування змінних, функцій, класів тощо.
-